

Tunstall Coding – Group G

Implementation, Analysis, and Comparison with Huffman Coding



Simona Cholakova (89231126)

Alja Eremič (89231124)

1. Introduction

This report presents an implementation and analysis of Tunstall coding, a variable-to-fixed source coding scheme. Unlike Huffman coding which maps fixed-length source symbols to variable-length codewords, Tunstall coding maps variable-length source phrases to fixed-length binary codewords. This fundamental difference makes Tunstall coding especially useful in systems where fixed-rate output is required.

The task involves the following steps:

- Construct a sequence of 1000 symbols from the alphabet {a, b, c, d} using a given probability distribution.
- Build a Tunstall codebook of 16 codewords using the greedy expansion algorithm.
- Encode the source sequence and compute average codeword length and compression efficiency.
- Decode the encoded codeword and check if it matches the initial message.
- Compare the results from Tunstall against Huffman coding.

We implemented our code in the Java programming language.

The full code together with this report can also be found in this GitHub repository:

<https://github.com/simona-cholakova/Tunstall-coding.git>

2. Problem Setup

2.1 Alphabet and Probability Distribution

The source alphabet has four symbols. The probability distribution is skewed, with 'd' being heavily dominant. This skewness is what makes compression **possible** and makes Tunstall coding particularly useful.

The probability distribution assigned to each symbol is:

Symbol	Probability
d	0.70
c	0.15
b	0.10
a	0.05

2.2 Source Sequence Generation

A sequence of 1000 symbols was generated using Java's Random class. The generation logic maps random integers in [0, 100) to symbols according to their cumulative probabilities:

```
for (int i = 0; i < 1000; i++) {  
    int a = rand.nextInt(101);  
    if (a < 70)        bigBoy = bigBoy.concat("d");  
    else if (a < 85)    bigBoy = bigBoy.concat("c");  
    else if (a < 95)    bigBoy = bigBoy.concat("b");  
    else                bigBoy = bigBoy.concat("a");  
}
```

The above codeblock correctly implements inverse transform sampling: since $P(d)=0.70$, integers 0–69 map to 'd'; since $P(c)=0.15$, integers 70–84 map to 'c'; and so on. The result is a 1000 character long string whose empirical symbol frequencies approximate the theoretical distribution. The first part of the generated sequence is shown below:

```
dacdbdddddbddcdcdaddbbcdcdccdddddcdbddbdddadadcabdddddccddd...
```

3. Tunstall Codebook Construction

3.1 Algorithm Overview

The Tunstall algorithm builds a codebook by iteratively expanding the most probable phrase in the current dictionary. It begins with the single-symbol alphabet as the initial codebook. At each iteration, the phrase with the highest probability is removed and expanded by its four extensions, one for each symbol in the alphabet and each assigned a probability equal to the parent's probability multiplied by the symbol's own probability.

This greedy expansion continues until the codebook reaches the desired size. For a fixed-length binary codeword of k bits, the codebook can hold exactly 2^k entries. Here $k = 4$, so the target size is $2^4 = 16$.

3.2 Implementation

The **codebook** is an `ArrayList` of **Pair** objects, where each **Pair** is a tuple of the form (**key**: `String`, **value**: `BigDecimal`). **Key** represents the **phrase** of the codeword and **value** represents the **probability** of the codeword. A helper `ArrayList` called **initial** holds the four original symbols and their probabilities.

Initialization: The codebook is seeded by copying all entries from **initial**.

Expansion loop: The loop runs until the codebook reaches the target size (16). At each step:

1. The pair with the highest probability is identified as **biggestPair**.
2. Four new pairs are generated for each symbol in **initial** by setting:

New Pair Key: **biggestPair.key** concatenated with the initial symbol's key.
("biggestPair.key + p.key")

New Pair Value: **biggestPair.value** multiplied by the initial symbol's value.
("p.value.multiply(biggestPair.value)")

3. **biggestPair** is removed from codebook. Four new codewords are added to the codebook.

```
ArrayList<Pair> codeBook = new ArrayList<>(initial);

while (codeBook.size() < 16) {
    Pair biggestPair = codeBook.get(0);
    for (Pair p : codeBook) {
        if (p.value.compareTo(biggestPair.value) > 0)
            biggestPair = p;
    }
    for (Pair p : initial) {
        Pair newPair = new Pair(
            biggestPair.key + p.key,
            p.value.multiply(biggestPair.value)
        );
        codeBook.add(newPair);
    }
    codeBook.remove(biggestPair);
}
```

****BigDecimal arithmetic is used throughout to avoid floating-point rounding errors that could affect the probability comparisons, especially for long phrases.**

****Code book is later stored into two separate hashMaps. “Encoding” hash map of form <String, Integer> and “Decoding” hash map of form <Integer, String >. This allows for faster and easier access.**

3.3 The Resulting Codebook

After the algorithm finishes, the 16 resulting phrases, their probabilities, and their 4-bit binary codewords are:

Index	Phrase	Probability	Codeword (binary)
0	c	0.15	0000
1	b	0.1	0001
2	a	0.05	0010
3	dc	0.105	0011
4	db	0.07	0100
5	da	0.035	0101
6	ddc	0.0735	0110
7	ddb	0.049	0111
8	dda	0.0245	1000
9	dddc	0.05145	1001
10	dddb	0.0343	1010
11	ddda	0.01715	1011
12	dddd	0.16807	1100
13	ddddc	0.036015	1101
14	ddddb	0.02401	1110
15	dddda	0.012005	1111

The codebook reveals the algorithm's behavior clearly: single-symbol entries survive for the rare symbols 'c', 'b', and 'a', while 'd' being the most probable gets expanded repeatedly, producing long chains of 'd's suffixed with a less-probable symbol (or five consecutive 'd's).

4. Encoding the Source Sequence

4.1 Encoding Logic

Encoding scans the source string left to right. Starting from the current position, the encoder attempts to match the longest possible prefix that exists in the codebook. If a match is found, its 4-bit binary index is appended to the output and the position advances by the length of the matched phrase. If no match is found (because the remaining suffix is shorter than the shortest codebook phrase), the leftover characters are encoded with the highest-probability codeword:

```

while (!stop) {
    test = "";
    for (int j = 0; j <= letter; j++) {
        if (count + j >= bigBoy.length()) continue;
        test = test + bigBoy.charAt(count + j);
    }
    if (encode.containsKey(test)) {
        encodedBin += String.format("%4s",
            Integer.toBinaryString(encode.get(test)))
            .replace(' ', '0');
        count = count + letter + 1;
        letter = lenOfBiggestCodeWord;
    } else {
        letter--;
        if (letter < 0) {
            dirty = bigBoy.substring(count);
            // encode remaining with highest-prob phrase
            encodedBin += String.format("%4s", ...).replace(' ', '0');
            stop = true;
        }
    }
    if (count >= bigBoy.length()) stop = true;
}

```

In the test run, the final unmatched suffix was "ddd", which does not appear in the codebook (only "dddd", "dddc", "dddb", "ddda", "ddc", "ddb", "dda", and "dc" exist for 'd'-dominated phrases). It was therefore encoded using the highest-probability phrase, which is "dddd" (index 12, codeword 1100).

4.2 Encoded Output

The beginning of the encoded binary stream is:

```

0101000001001100010001100011100010100001000000110110000011000110
0100101010110101001100100001110100001001001000010000011101110000
1100001101110111010100010000010011001000001000100010001101110111
...

```

Each group of 4 bits corresponds to one codebook entry.

5. Decoding and Verification

Decoding is the exact inverse of encoding. The binary stream is split into 4-bit groups, each converted to a decimal index, and the corresponding phrase is looked up in the **Decode** HashMap:

```

String[] decoding = encodedBin.split("(?<=\\G.{4})");

for (int j = 0; j < decoding.length; j++) {
    decoded += decode.get(Integer.parseInt(decoding[j], 2));
}

// Trim to original length (dirty-bit fix)
if (decoded.length() > bigBoy.length()) {
    decoded = decoded.substring(0, bigBoy.length());
}

```

The final trim handles the case where the last phrase decoded contains more characters than what was actually remaining in the original source. This arises because the trailing "ddd" was encoded with the phrase "dddd", which decoded to five characters when only three were needed. The trim ensures the decoded output has exactly the same length as the input.

The program verifies the result:

```
Decoding successful: TRUE
```

6. Compression Statistics

6.1 Average Phrase Length

The average phrase length L is the expected number of source symbols per codeword. It is computed as a probability-weighted sum over all codebook entries:

```
L = sum over all phrases p of: P(p) * |p|

L = 0.15*1 + 0.10*1 + 0.05*1 + 0.105*2 + 0.07*2 + 0.035*2
    + 0.0735*3 + 0.049*3 + 0.0245*3 + 0.05145*4 + 0.0343*4
    + 0.01715*4 + 0.16807*5 + 0.036015*5 + 0.02401*5 + 0.012005*5

L = 2.7731 symbols
```

This means each 4-bit codeword represents, on average, approximately 2.77 source symbols.

6.2 Average Bits per Source Symbol

Since each phrase maps to exactly 4 bits, the average number of bits required to represent one source symbol is:

```
bits/symbol = 4 bits per codeword / L symbols per codeword
              = 4 / 2.7731
              = 1.4424 bits/symbol
```

6.3 Source Entropy

The theoretical lower bound on average bits per symbol for lossless coding is the source entropy H , computed as:

```
H = -sum of P(x) * log2(P(x)) for all symbols x

H = -(0.70 * log2(0.70) + 0.15 * log2(0.15)
    + 0.10 * log2(0.10) + 0.05 * log2(0.05))

H = 1.3190 bits/symbol
```

6.4 Tunstall Efficiency

Coding efficiency measures how close the achieved compression rate is to the theoretical optimum:

```
Efficiency = H / (bits/symbol) * 100%
            = 1.3190 / 1.4424 * 100%
            = 91.45%
```

At 91.45% efficiency, Tunstall coding achieves very good compression, but still not optimal.

7. Comparison with Huffman Coding

7.1 Huffman Codes for the Alphabet

Huffman coding assigns variable-length codewords to individual symbols, with shorter codewords for more probable symbols. For the given distribution, the optimal Huffman code is:

Symbol	Probability	Huffman Code
d	0.70	0
c	0.15	10
b	0.10	110
a	0.05	111

The most frequent symbol 'd' gets a 1-bit code, 'c' gets 2 bits, and the two rarer symbols have 3-bit codes.

7.2 Huffman Compression Statistics

Expected length = 0.70*1 + 0.15*2 + 0.10*3 + 0.05*3 = 0.70 + 0.30 + 0.30 + 0.15 = 1.4500 bits/symbol	
Huffman encoded length (1000 symbols)	= 1461 bits
Original (ASCII)	= 8000 bits
Compression ratio vs ASCII	= 81.74%
Huffman efficiency = 1.3190 / 1.4500 * 100% = 90.97%	

7.3 Side-by-Side Comparison

Metric	Value
Average phrase length	2.7731 symbols
Average encoded length (Tunstall)	1.4424 bits/symbol
Source entropy	1.3190 bits/symbol
Tunstall efficiency	91.45%
Huffman average length	1.4500 bits/symbol
Huffman encoded length	1461 bits
Huffman compression vs ASCII	81.74%
Huffman efficiency	90.97%

Tunstall coding (1.4424 bits/symbol, 91.45% efficiency) slightly outperforms Huffman coding (1.4500 bits/symbol, 90.97% efficiency) on this source. Both schemes are very close to each other and to the entropy limit, but Tunstall achieves better rate due to its approach to encode multiple symbols at once. As the codebook size increases (larger k), Tunstall coding will approach entropy more.

8. Conclusion

This report has demonstrated the full pipeline of Tunstall coding: source generation, greedy codebook construction, encoding, decoding, and statistical analysis compared against Huffman coding.

The key findings are:

- The Tunstall algorithm correctly built a 16-entry codebook from a 4-symbol alphabet, producing phrases up to length 5 dominated by the most probable symbol 'd'.
- The average phrase length of 2.7731 symbols per 4-bit codeword yields an average rate of 1.4424 bits/symbol, achieving 91.45% of the theoretical entropy limit.
- Decoding was verified to be lossless.
- Tunstall (91.45%) outperforms Huffman (90.97%) on this source and the given probability distribution of the alphabet.

Both Tunstall and Huffman coding are practical, nearly optimal lossless compression schemes. The choice between them depends on the application: Tunstall's fixed-rate output suits constrained hardware and communication channels, while Huffman's simplicity and streaming nature make it preferable in general-purpose software compression. For sources with more extreme skewness or larger alphabets, arithmetic coding or asymmetric numeral systems would be considered as even closer approaches to the entropy limit.